Geth 기본 실행 흐름 분석

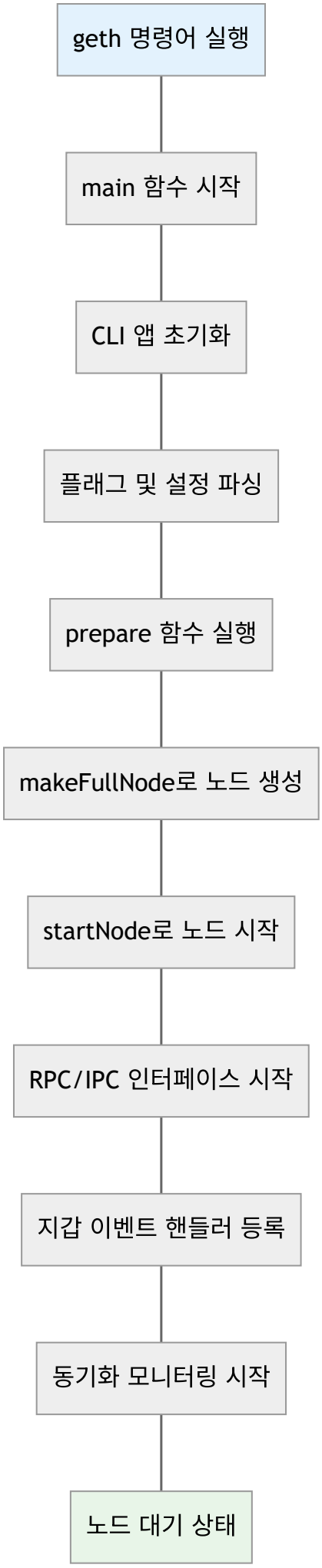
`geth` 명령어만 입력했을 때 발생하는 내부 동작 과정을 단계별로 분석한 문서입니다.

> **분석 대상:** `geth` (인수 없이 실행)

> **진입점:** `main()` → `geth()` 함수

> **소스 파일:** `cmd/geth/main.go`

🚀 전체 실행 흐름 개요


geth 명령어 실행
main 함수 시작
CLI 앱 초기화
플래그 및 설정 파싱
prepare 함수 실행
makeFullNode로 노드 생성
startNode로 노드 시작
RPC/IPC 인터페이스 시작
지갑 이벤트 핸들러 등록
동기화 모니터링 시작
노드 대기 상태

📋 단계별 상세 분석

1단계: 프로그램 진입점 (`main()`)

```
func main() {
if err := app.Run(os.Args); err != nil {
fmt.Fprintln(os.Stderr, err)
os.Exit(1)
}
}
```

동작 내용:

- **CLI 애플리케이션 실행**: `app.Run(os.Args)` 로 urfave/cli 프레임워크 시작
- **명령줄 인수 파싱**: `os.Args` 에서 플래그와 명령어 추출
- **오류 발생 시 프로그램 종료**: 파싱 오류나 실행 오류 시 stderr에 출력 후 종료

핵심 포인트:

- `app` 은 전역 변수로 `init()` 함수에서 초기화됨
- `os.Args[0]` 은 프로그램명, `os.Args[1:]` 이 실제 인수들
- 오류 시 exit code 1로 종료하여 스크립트에서 실패 감지 가능

2단계: CLI 앱 초기화 (`init()`)

앱 기본 설정:

```
var app = flags.NewApp("the go-ethereum command line interface")
```

```
func  init() {
// 기본 액션을 geth 함수로 설정 (인수 없이 실행 시)
app.Action = geth
// 사용 가능한 모든 명령어 등록
app.Commands = []*cli.Command{
// 체인 관련 명령어
initCommand, importCommand, exportCommand,
// 계정 관련 명령어
accountCommand, walletCommand,
// 콘솔 관련 명령어
consoleCommand, attachCommand, javascriptCommand,
// 기타 명령어
versionCommand, licenseCommand,
dbCommand, snapshotCommand,
}
// 모든 플래그 통합
app.Flags = slices.Concat(
nodeFlags, // 노드 설정 플래그
rpcFlags, // RPC 인터페이스 플래그
consoleFlags, // 콘솔 플래그
debug.Flags, // 디버그 플래그
metricsFlags, // 메트릭 플래그
)
```

```
// 환경 변수 자동 매핑 (GETH_ 접두사)
flags.AutoEnvVars(app.Flags, "GETH")
}
```

Before 훅 실행 (모든 명령어 실행 전):

```
app.Before = func(ctx *cli.Context) error {
// 1. CPU 코어 수 자동 설정 (컨테이너 환경 고려)
maxprocs.Set()
// 2. 레거시 플래그 마이그레이션
flags.MigrateGlobalFlags(ctx)
// 3. 디버그 설정 (로깅, 프로파일링 등)
if err := debug.Setup(ctx); err != nil {
return err
}
// 4. 환경 변수 검증
flags.CheckEnvVars(ctx, app.Flags, "GETH")
return nil
}
```

핵심 포인트:

- **명령어 우선순위**: 특정 명령어가 없으면 `app.Action` (geth 함수) 실행
- **플래그 카테고리**: 총 130개 이상의 플래그를 5개 카테고리로 분류
- **환경 변수**: `GETH_CACHE=4096` → `--cache 4096` 과 동일

- **자동 설정**: 컨테이너 환경에서 CPU 제한 자동 감지

3단계: 기본 액션 실행 (`geth()`)

```go
// geth는 특별한 서브명령어가 실행되지 않을 때의 메인 진입점입니다.
// 명령줄 인수를 기반으로 기본 노드를 생성하고 블로킹 모드로 실행하여
// 종료될 때까지 대기합니다.
func geth(ctx *cli.Context) error {
// 1. 인수 검증: 추가 인수가 있으면 오류
if args := ctx.Args().Slice(); len(args) > 0 {
return fmt.Errorf("invalid command: %q", args[0])
}

// 2. 노드 시작 전 준비 작업
prepare(ctx)
// 3. 풀 노드 생성 (P2P, 블록체인, RPC 등 모든 컴포넌트)
stack := makeFullNode(ctx)
defer stack.Close() // 함수 종료 시 리소스 정리
// 4. 노드 시작 (네트워크 연결, 서비스 활성화)
startNode(ctx, stack, false) // isConsole = false
// 5. 노드가 종료될 때까지 대기 (블로킹)
stack.Wait()
return nil
```

```
}
```

동작 순서 상세:

1. **인수 검증**: `geth extra_arg` 같은 잘못된 사용 방지
2. **준비 작업**: 네트워크 감지, 캐시 최적화, 로깅 설정
3. **노드 생성**: 설정 로드 → 스택 생성 → 서비스 등록
4. **노드 시작**: P2P 시작 → RPC 활성화 → 지갑 관리 → 동기화 시작
5. **대기 상태**: 시그널 대기, 정상 종료 시까지 블로킹

핵심 포인트:

- **블로킹 실행**: `stack.Wait()` 에서 SIGINT/SIGTERM 대기
- **리소스 관리**: `defer stack.Close()` 로 안전한 정리 보장
- **오류 처리**: 각 단계에서 오류 발생 시 즉시 반환

4단계: 준비 작업 (`prepare()`)

```
// prepare는 메모리 캐시 할당량을 조작하고 메트릭 시스템을 설정합니다.
// 이 함수는 devp2p 스택을 시작하기 전에 호출되어야 합니다.
func prepare(ctx *cli.Context) {
// 1. 네트워크 감지 및 로깅
switch {
case ctx.IsSet(utils.SepoliaFlag.Name):
log.Info("Starting Geth on Sepolia testnet...")
case ctx.IsSet(utils.HoleskyFlag.Name):
log.Info("Starting Geth on Holesky testnet...")
```

```
case  ctx.IsSet(utils.HoodiFlag.Name):

log.Info("Starting Geth on Hoodi testnet...")

case !ctx.IsSet(utils.NetworkIdFlag.Name):

log.Info("Starting Geth on Ethereum mainnet...")

}

// 2. 메인넷 캐시 자동 최적화

if !ctx.IsSet(utils.CacheFlag.Name) && !ctx.IsSet(utils.NetworkIdFlag.Name) {

// 메인넷이고 캐시가 명시적으로 설정되지 않은 경우

if !ctx.IsSet(utils.HoleskyFlag.Name) &&

!ctx.IsSet(utils.SepoliaFlag.Name) &&

!ctx.IsSet(utils.HoodiFlag.Name) &&

!ctx.IsSet(utils.DeveloperFlag.Name) {

// 메인넷 확인 후 캐시 증가

log.Info("Bumping default cache on mainnet",

"provided", ctx.Int(utils.CacheFlag.Name),

"updated", 4096)

ctx.Set(utils.CacheFlag.Name, strconv.Itoa(4096))

}

}

}
```

동작 상세:

1. 네트워크 자동 감지:

- **플래그 우선순위**: 명시적 네트워크 플래그 → 기본 메인넷
- **로깅**: 사용자에게 어떤 네트워크에 연결하는지 명확히 표시
- **설정 검증**: 여러 네트워크 플래그 동시 설정 방지

2. 메인넷 성능 최적화:

```
// 기본 캐시: 1024MB → 메인넷 자동 증가: 4096MB
// 이유: 메인넷은 상태 크기가 크므로 더 많은 캐시 필요
if 메인넷 && 캐시_미설정 && 테스트넷_아님 && 개발모드_아님 {
캐시 = 4096MB // 4배 증가
}
```

핵심 포인트:

- **자동 튜닝**: 사용자가 설정하지 않아도 최적 성능 제공
- **네트워크별 차별화**: 메인넷은 더 많은 리소스 할당
- **명시적 설정 존중**: 사용자가 직접 설정한 값은 변경하지 않음
- **개발자 친화적**: 개발 모드에서는 가벼운 설정 유지

5단계: 풀 노드 생성 (`makeFullNode()`)

```
// makeFullNode는 geth 설정을 로드하고 Ethereum 백엔드를 생성합니다.
func makeFullNode(ctx *cli.Context) *node.Node {
// 1. 설정 노드 생성 (기본 스택 + 설정)
stack, cfg := makeConfigNode(ctx)
// 2. 하드포크 오버라이드 설정
if ctx.IsSet(utils.OverrideOsaka.Name) {
```

```
v := ctx.Uint64(utils.OverrideOsaka.Name)
cfg.Eth.OverrideOsaka = &v
}
if ctx.IsSet(utils.OverrideVerkle.Name) {
v := ctx.Uint64(utils.OverrideVerkle.Name)
cfg.Eth.OverrideVerkle = &v
}

// 3. 메트릭 시스템 시작
utils.SetupMetrics(&cfg.Metrics)

// 4. Ethereum 서비스 등록 (핵심 블록체인 기능)
backend, eth := utils.RegisterEthService(stack, &cfg.Eth)

// 5. 시스템 정보 메트릭 등록
if eth != nil {
var protos []string
for _, p := range eth.Protocols() {
protos = append(protos, fmt.Sprintf("%v/%d", p.Name, p.Version))
}
metrics.NewRegisteredGaugeInfo("geth/info", nil).Update(metrics.GaugeInfoValue{
"arch": runtime.GOARCH,
"os": runtime.GOOS,
```

```
"version": cfg.Node.Version,
"protocols": strings.Join(protos, ","),
})
}

// 6. 로그 필터 RPC API 설정
filterSystem := utils.RegisterFilterAPI(stack, backend, &cfg.Eth)

// 7. GraphQL 서비스 등록 (선택적)
if ctx.IsSet(utils.GraphQLEnabledFlag.Name) {
utils.RegisterGraphQLService(stack, backend, filterSystem, &cfg.Node)
}
// 8. Ethereum Stats 데몬 등록 (선택적)
if cfg.Ethstats.URL != "" {
utils.RegisterEthStatsService(stack, backend, cfg.Ethstats.URL)
}

return stack
}
```

makeConfigNode 함수 (기본 스택 생성):

```
func makeConfigNode(ctx *cli.Context) (*node.Node, gethConfig) {
```

```
// 1. 기본 설정 로드
cfg := loadBaseConfig(ctx)
// 2. 노드 스택 생성
stack, err := node.New(&cfg.Node)
if err != nil {
utils.Fatalf("Failed to create the protocol stack: %v", err)
}
// 3. 계정 매니저 백엔드 설정
if err := setAccountManagerBackends(stack.Config(),
stack.AccountManager(),
stack.KeyStoreDir()); err != nil {
utils.Fatalf("Failed to set account manager backends: %v", err)
}


// 4. Ethereum 설정 적용
utils.SetEthConfig(ctx, stack, &cfg.Eth)
return stack, cfg
}
```

생성되는 주요 컴포넌트:

1. 노드 스택 (`*node.Node`)

- P2P 네트워크 레이어
- RPC 서버 (HTTP, WebSocket, IPC)

- 서비스 레지스트리
- 계정 매니저

2. Ethereum 백엔드 (`*eth.Ethereum`)

- 블록체인 데이터베이스
- 트랜잭션 풀
- 마이너 (선택적)
- 동기화 엔진

3. 추가 서비스들

- 로그 필터 시스템
- GraphQL API (선택적)
- 메트릭 수집기
- Stats 리포터 (선택적)

핵심 포인트:

- **모듈식 설계**: 각 기능이 독립적인 서비스로 등록
- **설정 기반**: CLI 플래그에 따라 서비스 활성화/비활성화
- **리소스 관리**: 스택이 모든 서비스의 생명주기 관리
- **확장성**: 새로운 서비스를 쉽게 추가 가능

6단계: 노드 시작 (`startNode()`)

```
// startNode는 시스템 노드와 등록된 모든 프로토콜을 부팅한 후
// RPC/IPC 인터페이스와 마이너를 시작합니다.
func startNode(ctx *cli.Context, stack *node.Node, isConsole bool) {
// 1. 노드 자체 시작
```

```
utils.StartNode(ctx, stack, isConsole)


// 2. 지갑 이벤트 핸들러 등록
events := make(chan accounts.WalletEvent, 16)
stack.AccountManager().Subscribe(events)


// 3. 로컬 geth 노드와 상호작용할 클라이언트 생성
rpcClient := stack.Attach()
ethClient := ethclient.NewClient(rpcClient)


// 4. 지갑 관리 고루틴 시작
go func() {
// 이미 연결된 지갑들 열기
for _, wallet := range stack.AccountManager().Wallets() {
if err := wallet.Open(""); err != nil {
log.Warn("Failed to open wallet", "url", wallet.URL(), "err", err)
}
}
// 지갑 이벤트 처리 루프
for event := range events {
switch event.Kind {
case accounts.WalletArrived:
// 새 지갑 감지 시 자동 열기
```

```
if err := event.Wallet.Open(""); err != nil {
log.Warn("New wallet appeared, failed to open",
"url", event.Wallet.URL(), "err", err)
}
case accounts.WalletOpened:
// 지갑 열림 시 상태 로깅 및 자동 파생
status, _ := event.Wallet.Status()
log.Info("New wallet appeared",
"url", event.Wallet.URL(), "status", status)

var derivationPaths []accounts.DerivationPath
if event.Wallet.URL().Scheme == "ledger" {
derivationPaths = append(derivationPaths,
accounts.LegacyLedgerBaseDerivationPath)
}
derivationPaths = append(derivationPaths,
accounts.DefaultBaseDerivationPath)

event.Wallet.SelfDerive(derivationPaths, ethClient)

case accounts.WalletDropped:
// 지갑 제거 시 정리
log.Info("Old wallet dropped", "url", event.Wallet.URL())
```

```
event.Wallet.Close()
}
}
}()

// 5. 동기화 완료 시 자동 종료 모니터링 (선택적)
if ctx.Bool(utils.ExitWhenSyncedFlag.Name) {
go func() {
sub := stack.EventMux().Subscribe(downloader.DoneEvent{})
defer sub.Unsubscribe()
for {
event := <-sub.Chan()
if event == nil {
continue
}
done, ok := event.Data.(downloader.DoneEvent)
if !ok {
continue
}
// 최신 블록이 10분 이내인지 확인
if timestamp := time.Unix(int64(done.Latest.Time), 0);
time.Since(timestamp) < 10*time.Minute {
log.Info("Synchronisation completed",
```

```
"latestnum", done.Latest.Number,
"latesthash", done.Latest.Hash(),
"age", common.PrettyAge(timestamp))
stack.Close()
}
}
}()
}
}
```

utils.StartNode 함수 상세:

```
func StartNode(ctx *cli.Context, stack *node.Node, isConsole bool) {
// 1. 스택 시작 (모든 등록된 서비스 활성화)
if err := stack.Start(); err != nil {
Fatalf("Error starting protocol stack: %v", err)
}
// 2. 시그널 핸들러 고루틴 시작
go func() {
sigc := make(chan os.Signal, 1)
signal.Notify(sigc, syscall.SIGINT, syscall.SIGTERM)
defer signal.Stop(sigc)
```

```
// 3. 디스크 공간 모니터링 (선택적)

minFreeDiskSpace := 2 * ethconfig.Defaults.TrieDirtyCache

if ctx.IsSet(MinFreeDiskSpaceFlag.Name) {

minFreeDiskSpace = ctx.Int(MinFreeDiskSpaceFlag.Name)

}

if minFreeDiskSpace > 0 {

go monitorFreeDiskSpace(sigc, stack.InstanceDir(),

uint64(minFreeDiskSpace)*1024*1024)

}



// 4. 종료 처리 함수

shutdown := func() {

log.Info("Got interrupt, shutting down...")

go stack.Close()

// 10번의 추가 인터럽트까지 대기

for i := 10; i > 0; i-- {

<-sigc

if i > 1 {

log.Warn("Already shutting down, interrupt more to panic.",

"times", i-1)

}

}

debug.Exit() // 트레이스 및 CPU 프로파일 데이터 플러시
```

```
debug.LoudPanic("boom")

}



// 5. 콘솔 모드와 일반 모드 구분

if  isConsole {

// 콘솔 모드: SIGINT는 무시, SIGTERM만 처리

for {

sig := <-sigc

if  sig == syscall.SIGTERM {

shutdown()

return

}

}

} else {

// 일반 모드: 첫 번째 시그널에서 종료

<-sigc

shutdown()

}

}()

}
```

시작되는 주요 서비스들:

1. P2P 네트워크

- 부트노드 연결
- 피어 디스커버리 시작
- 프로토콜 핸드셰이크

2. RPC 서버들

- HTTP RPC (기본 포트: 8545)
- WebSocket RPC (기본 포트: 8546)
- IPC RPC (파이프/소켓)

3. 블록체인 서비스

- 블록 동기화 시작
- 트랜잭션 풀 활성화
- 상태 데이터베이스 로드

4. 지갑 관리

- 하드웨어 지갑 감지
- 키스토어 모니터링
- 자동 계정 파생

핵심 포인트:

- **비동기 시작**: 모든 서비스가 고루틴으로 병렬 시작
- **우아한 종료**: 시그널 처리로 안전한 종료 보장
- **리소스 모니터링**: 디스크 공간 부족 시 자동 종료
- **콘솔 모드 지원**: 대화형 모드에서 다른 시그널 처리

7단계: 서비스 실행 상태

노드 대기 및 서비스 운영:

```
// geth() 함수의 마지막 단계
startNode(ctx, stack, false)
stack.Wait() // 여기서 블로킹되어 종료 시그널 대기
return nil
```

stack.Wait() 내부 동작:

```
// node/node.go의 Wait 메서드
func (n *Node) Wait() {
<-n.stop  // stop 채널에서 신호 대기
}

// 종료 시그널 처리 (utils.StartNode의 고루틴에서)
shutdown := func() {
log.Info("Got interrupt, shutting down...")
go stack.Close() // stop 채널에 신호 전송
// ... 추가 종료 처리
}
```

활성화되는 주요 서비스들:

1. P2P 네트워크 레이어

```
// 부트노드 연결 및 피어 디스커버리
- 하드코딩된 부트노드 목록에 연결
```

```
- DHT 기반 피어 디스커버리 시작
- 최대 50개 피어 연결 유지 (기본값)
- eth/68, snap/1 프로토콜 지원
```

2. RPC 인터페이스 서버

```
// HTTP RPC (포트 8545, 기본 비활성화)
if ctx.IsSet(utils.HTTPEnabledFlag.Name) {
// JSON-RPC 2.0 over HTTP
// CORS, 가상 호스트 검증
}

// WebSocket RPC (포트 8546, 기본 비활성화)
if ctx.IsSet(utils.WSEnabledFlag.Name) {
// 실시간 이벤트 구독 지원
}

// IPC RPC (항상 활성화)
// Unix 소켓 또는 Named Pipe 사용
```

3. 블록체인 동기화 엔진

```
// 동기화 모드별 동작
switch syncMode {
```

```
case downloader.SnapSync: // 기본값
// 최신 상태 스냅샷 다운로드
// 이후 블록 헤더부터 순차 동기화
case downloader.FastSync:
// 블록 헤더 + 영수증 다운로드
// 상태는 최신 블록에서만 검증
case downloader.FullSync:
// 모든 블록과 트랜잭션 완전 검증
// 가장 안전하지만 가장 느림
}
```

4. 트랜잭션 풀 관리

```
// 트랜잭션 수집 및 검증
- 네트워크에서 트랜잭션 수신
- 가스 가격, 논스 검증
- 계정별 큐 관리 (기본: 16개/계정)
- 전체 풀 크기 제한 (기본: 4096개)
```

5. 상태 데이터베이스

```
// 계정 상태 관리
- 계정 잔액, 논스, 코드 저장
- 머클 패트리샤 트라이 구조
- 스냅샷 기반 빠른 접근
```

```
- LevelDB 백엔드 사용
```

6. 지갑 및 계정 관리

```
// 자동 지갑 감지 및 관리
for event := range walletEvents {
switch event.Kind {
case accounts.WalletArrived:
// USB 하드웨어 지갑 연결 감지
// 키스토어 파일 변경 감지
case accounts.WalletOpened:
// HD 지갑 자동 계정 파생
// Ledger, Trezor 지원
}
}
```

실시간 모니터링 지표:

```
// 로그 출력 예시
INFO [날짜] Starting Geth on Ethereum mainnet...
INFO [날짜] Bumping default cache on mainnet provided=1024 updated=4096
INFO [날짜] Starting P2P networking
INFO [날짜] Started P2P networking self=enode://...
INFO [날짜] IPC endpoint opened url=\\\\.\\pipe\\geth.ipc
```

```
INFO [날짜] Looking for peers peercount=0 tried=0 static=0

INFO [날짜] Block synchronisation started

INFO [날짜] Imported new chain segment blocks=1 txs=0 mgas=0.000
```

핵심 포인트:

- **이벤트 기반**: 모든 서비스가 이벤트 루프로 동작
- **비동기 처리**: 블록 동기화, 트랜잭션 처리가 병렬 실행
- **자동 복구**: 네트워크 연결 끊김 시 자동 재연결
- **리소스 관리**: 메모리, 디스크 사용량 지속 모니터링
- **우아한 종료**: SIGINT/SIGTERM 시 모든 서비스 안전 종료

🔧 주요 구성 요소

플래그 카테고리

| 카테고리 | 설명 | 주요 플래그 |

|---|---|---|

| **nodeFlags** | 노드 기본 설정 | `--identity`, `--bootnodes`, `--cache` |

| **rpcFlags** | RPC 인터페이스 | `--http`, `--ws`, `--ipc` |

| **metricsFlags** | 메트릭 수집 | `--metrics`, `--metrics.port` |

| **debug.Flags** | 디버깅 옵션 | `--verbosity`, `--log.debug` |

기본 설정값

```
const clientIdentifier = "geth"
```

- **클라이언트 식별자**: “geth”
- **기본 네트워크**: Ethereum 메인넷
- **기본 캐시**: 메인넷에서 4096MB, 테스트넷에서 더 적음
- **기본 포트**: P2P 30303, HTTP RPC 8545

🌐 네트워크 연결 과정

1. 부트노드 연결

- 하드코딩된 부트노드 목록 사용
- DNS 디스커버리 활성화 (기본값)
- 피어 검색 시작

2. 블록체인 동기화

- **Fast Sync** (기본값): 최신 상태만 다운로드
- **Full Sync**: 모든 블록 검증
- **Snap Sync**: 스냅샷 기반 빠른 동기화

3. 트랜잭션 처리

- 트랜잭션 풀 초기화
- 네트워크에서 트랜잭션 수신
- 블록 생성 및 전파 (마이닝 활성화 시)

📊 리소스 사용량

메모리 사용

- **기본 캐시**: 1024MB (테스트넷), 4096MB (메인넷)
- **트라이 캐시**: 전체 캐시의 25%
- **데이터베이스 캐시**: 전체 캐시의 50%

디스크 사용

- **체인데이터**: 수백 GB (메인넷 풀 노드)
- **상태 데이터**: 수십 GB
- **로그 파일**: 설정에 따라 가변

네트워크 사용

- **초기 동기화**: 수백 GB 다운로드
- **일반 운영**: 지속적인 블록/트랜잭션 동기화

🛠️ 문제 해결

일반적인 시작 문제

1. **포트 충돌**

```
WARN [날짜] Failed to start HTTP server: listen tcp :8545: bind: address already
```

해결: `--http.port` 플래그로 다른 포트 사용

2. **디스크 공간 부족**

```
ERROR [날짜] Failed to write block: not enough disk space
```

해결: 디스크 공간 확보 또는 `--cache` 값 조정

3. **네트워크 연결 문제**

```
WARN [날짜] No peers available for sync
```

해결: 방화벽 설정 확인, 부트노드 연결 상태 점검

📈 성능 최적화 팁

1. 캐시 설정

```
geth  --cache  8192  # 8GB 캐시 할당
```

2. 동기화 모드

```
geth  --syncmode  snap  # 빠른 스냅샷 동기화
```

3. 가비지 컬렉션

```
geth  --cache.gc  25  # GC 트리거 임계값 설정
```

🔚 종료 과정

시그널 기반 정상 종료 (SIGTERM/SIGINT)

1. 시그널 감지 및 종료 시작:

```
// utils.StartNode의 시그널 핸들러에서
shutdown := func() {
log.Info("Got interrupt, shutting down...")
go stack.Close() // 비동기로 종료 시작
// 추가 인터럽트 처리 (강제 종료 방지)
for i := 10; i > 0; i-- {
<-sigc
if i > 1 {
log.Warn("Already shutting down, interrupt more to panic.",
"times", i-1)
}
}
debug.Exit() // 프로파일 데이터 플러시
debug.LoudPanic("boom") // 최후 수단
}
```

2. 스택 종료 과정 (`stack.Close()`):

```
// node/node.go의 Close 메서드
func (n *Node) Close() error {
// 1. 중복 종료 방지
n.lock.Lock()
defer n.lock.Unlock()
```

```
if n.state == closedState {
return ErrNodeStopped
}
// 2. 모든 등록된 서비스 역순으로 종료
for _, service := range n.services {
if err := service.Stop(); err != nil {
log.Error("Failed to stop service", "service", service, "err", err)
}
}
// 3. P2P 서버 종료
if n.server != nil {
n.server.Stop()
}
// 4. RPC 서버들 종료
n.stopRPC()
// 5. 데이터베이스 안전 종료
if n.database != nil {
n.database.Close()
}
// 6. 대기 중인 고루틴들에게 종료 신호
close(n.stop)
n.state = closedState
return nil
```

```
}
```

3. 서비스별 종료 순서:

Ethereum 서비스 종료:

```
// eth/backend.go의 Stop 메서드
func (s *Ethereum) Stop() error {
// 1. 마이너 중지 (있는 경우)
if  s.miner != nil {
s.miner.Close()
}
// 2. 블록체인 동기화 중지
s.downloader.Terminate()
// 3. 트랜잭션 풀 종료
s.txPool.Stop()
// 4. 블록체인 데이터베이스 종료
s.blockchain.Stop()
// 5. 이벤트 구독 정리
s.eventMux.Stop()
return  nil
}
```

P2P 네트워크 종료:

```
// p2p/server.go의 Stop 메서드
```

```
func (srv *Server) Stop() {
// 1. 새로운 연결 거부
srv.lock.Lock()
if !srv.running {
srv.lock.Unlock()
return
}
srv.running = false
srv.lock.Unlock()
// 2. 모든 활성 피어 연결 종료
for _, peer := range srv.peers {
peer.Disconnect(DiscQuitting)
}
// 3. 리스너 소켓 닫기
if srv.listener != nil {
srv.listener.Close()
}
// 4. 디스커버리 테이블 정리
if srv.ntab != nil {
srv.ntab.Close()
}
}
```

4. 데이터베이스 안전 종료:

```
// core/blockchain.go의 Stop 메서드
func (bc *BlockChain) Stop() {
// 1. 진행 중인 블록 처리 완료 대기
bc.wg.Wait()
// 2. 캐시 플러시
bc.stateCache.TrieDB().Close()
// 3. 데이터베이스 커밋
bc.db.Close()
log.Info("Blockchain stopped")
}
```

5. After 훅 실행:

```
// main.go의 init()에서 설정된 After 훅
app.After = func(ctx *cli.Context) error {
// 1. 디버그 시스템 정리 (프로파일 저장 등)
debug.Exit()
// 2. 터미널 모드 리셋 (콘솔 모드에서 중요)
prompt.Stdin.Close()
return  nil
}
```

종료 로그 예시:

```
INFO [날짜] Got interrupt, shutting down...

INFO [날짜] Ethereum protocol stopped

INFO [날짜] Transaction pool stopped

INFO [날짜] Blockchain stopped

INFO [날짜] Database closed

INFO [날짜] P2P server stopped

INFO [날짜] RPC server stopped
```

핵심 포인트:

- **우아한 종료**: 진행 중인 작업 완료 후 종료
- **데이터 무결성**: 데이터베이스 커밋 후 안전 종료
- **리소스 정리**: 모든 네트워크 연결, 파일 핸들 정리
- **강제 종료 방지**: 10번의 추가 인터럽트까지 대기
- **복구 가능**: 다음 시작 시 일관된 상태에서 재개

참고 자료:

- Geth 공식 문서
- Go-Ethereum GitHub
- Ethereum 개발자 문서

이 문서는 go-ethereum 소스 코드 분석을 바탕으로 작성되었습니다.